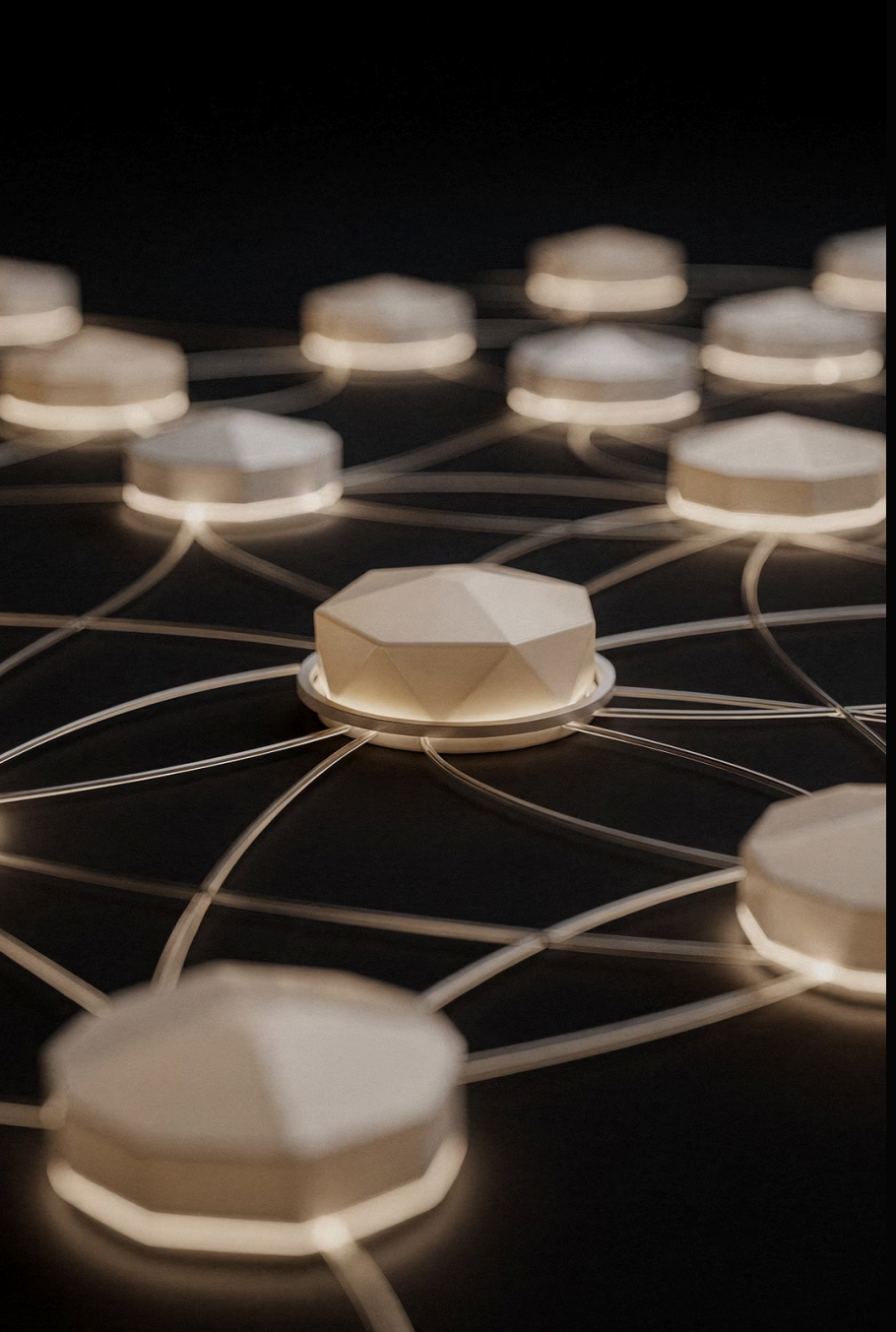


Microserviço

Objetivo: Entender o que é uma arquitetura de microserviços, por que ela existe, quais problemas ela resolve (e quais ela cria), e os principais conceitos que você precisa conhecer para trabalhar com ela.

> O **vendas-ms** que você está construindo **é** um microserviço. O sufixo `-ms` não é coincidência — é a abreviação de *microservice*. Ao final deste material você vai entender o que isso significa e quais são as implicações.

1 — O ponto de partida: o Monólito

Um **monólito** é uma aplicação em que todas as funcionalidades — clientes, pedidos, pagamentos, estoque, notificações — são desenvolvidas, implantadas e executadas como **uma única unidade**. Um processo, memória compartilhada, mesmo banco de dados, mesmo build e deploy.

Problemas do monólito em escala

Deploy acoplado

Corrigir um bug no módulo de clientes exige deploy da aplicação inteira.

Escalabilidade engessada

Se pedidos precisa de mais CPU, você escala tudo — inclusive módulos que não precisam.

Time bomb tecnológica

Toda a aplicação usa a mesma versão de Java, Spring, Hibernate — actualizar uma dependência afeta tudo.

Falha catastrófica

Um vazamento de memória no módulo de relatórios pode derrubar pedidos e pagamentos.

Atenção: monólito **não é errado**. Para equipes pequenas e aplicações jovens, é frequentemente a escolha certa. Comece simples, decomponha quando a dor justificar o custo.

2 — Microserviços: a solução (e seus trade-offs)

Um **microserviço** é uma aplicação pequena e autónoma responsável por um domínio específico do negócio, implantada independentemente, que se comunica pela rede e possui seu próprio banco de dados.

> O **vendas-ms** é o serviço `pedidos-ms` desse ecossistema: responsável por clientes e pedidos de vendas, implantado de forma independente, com seu próprio banco de dados MySQL.

Monólito vs Microserviços

Aspecto	Monólito	Microserviços
Deploy	Uma unidade, tudo junto	Cada serviço independentemente
Escalabilidade	Toda a aplicação	Apenas os serviços que precisam
Banco de dados	Compartilhado	Um por serviço
Comunicação interna	Chamada de método (memória)	HTTP ou mensageria (rede)
Falha isolada	Um módulo pode derrubar tudo	Falha não necessariamente propaga
Complexidade	Baixa operacionalmente	Alta — requer orquestração e observabilidade
Recomendado para	Times pequenos, produto em validação	Produtos maduros, times grandes, escala

3 — Conceitos fundamentais

3.1 Domínio e Bounded Context

O que é Bounded Context?

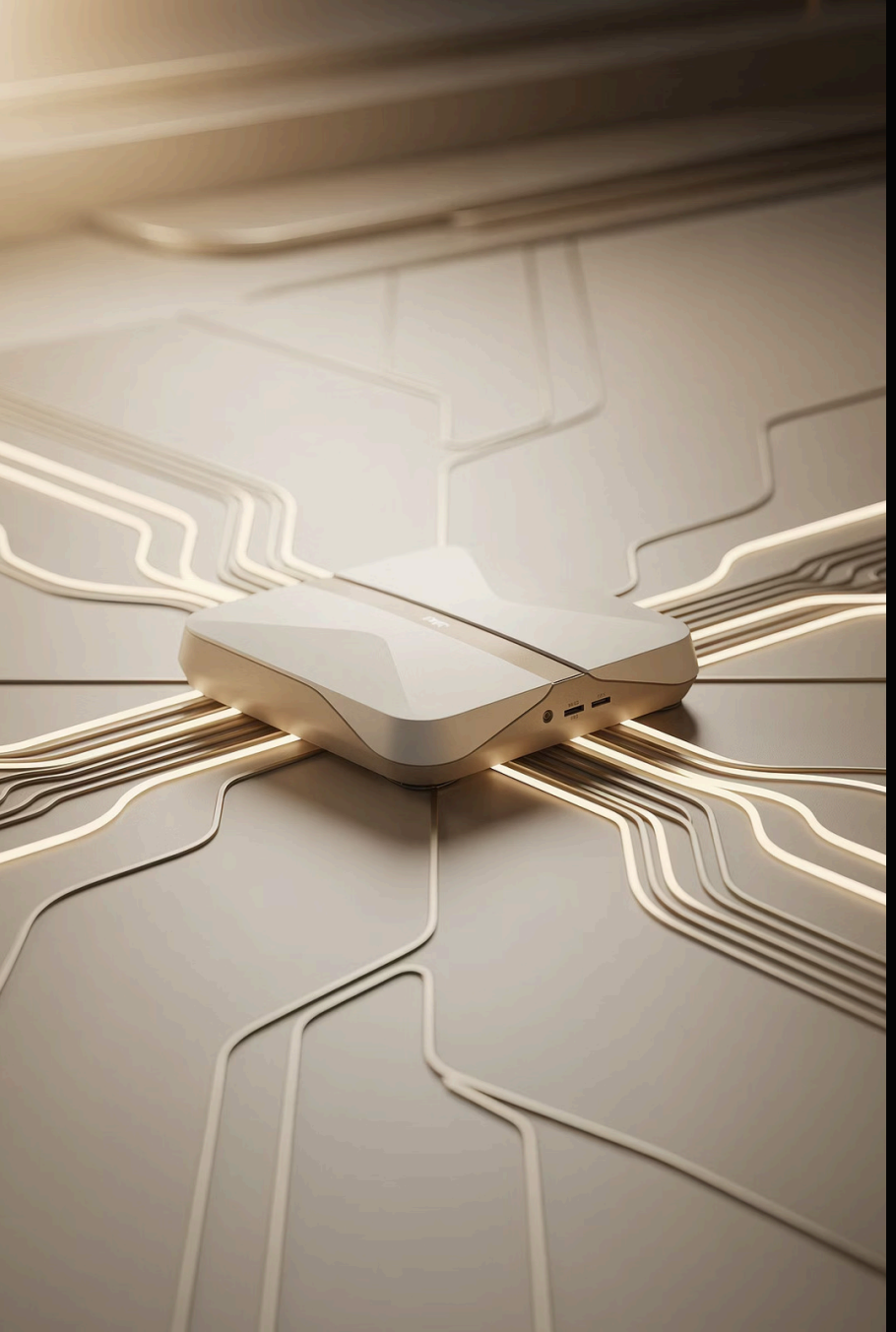
Conceito do **Domain-Driven Design (DDD)**: um microserviço deve ser responsável por uma fronteira clara dentro do domínio onde os conceitos têm significado único e consistente.

Cada serviço tem sua **própria definição** dos dados — não compartilham a mesma tabela.

Exemplo: o que é "cliente"?

- **Vendas:** quem fez um pedido — CPF, nome, endereço de entrega
- **Marketing:** um lead com histórico de campanhas e score de engajamento
- **Financeiro:** um devedor com limite de crédito e histórico de pagamentos

Regra prática: se dois serviços precisam dos mesmos dados, eles comunicam-se via API — mas cada um armazena os dados que lhe interessam em seu próprio banco.



3.2 API Gateway

Com múltiplos serviços em endereços diferentes, o cliente (browser, app mobile) não pode conhecer o endereço de cada um. O **API Gateway** é o ponto de entrada único que recebe todas as requisições, roteia para o serviço correcto, e pode aplicar autenticação, rate limiting e logging de forma centralizada.

Spring Cloud Gateway

Para ecossistemas Spring Boot

Kong

Independente de linguagem, altamente configurável

AWS API Gateway

Gerenciado na nuvem AWS

NGINX

Proxy reverso que pode funcionar como gateway simples

3.3 Comunicação entre Serviços

Síncrona (HTTP/REST)

Um serviço chama o outro directamente e **espera a resposta** antes de continuar.

Ferramentas: RestTemplate, WebClient, **OpenFeign** (já usado para chamar o ViaCEP).

✅ Simples de implementar e debugar.
❌ Se `clientes-ms` está fora do ar, `pedidos-ms` também falha — **acoplamento temporal**.

Assíncrona (Mensageria)

Um serviço publica uma mensagem num **broker** e continua. O outro consome quando disponível.

Ferramentas: Apache Kafka, RabbitMQ, AWS SQS/SNS.

✅ Desacoplamento total — se `estoque-ms` cair, o pedido ainda é criado.
❌ Mais complexo; consistência é **eventual**, não imediata.

Modelo	Acoplamento	Consistência	Quando usar
Request/Response	Alto — emissor espera receptor	Imediata	Quando precisa da resposta para continuar
Publish/Subscribe	Baixo — receptor pode estar offline	Eventual	Quando a operação pode ser processada depois

3.4 Service Discovery

Em ambientes com containers que sobem e caem dinamicamente, os endereços IP não são fixos. O **Service Discovery** resolve isso:

- Cada serviço ao iniciar **regista-se** num servidor central com seu nome e endereço
- Quando um serviço quer chamar outro, **consulta** o servidor para descobrir o endereço actual

Ferramentas

Netflix Eureka

Spring Cloud

Consul

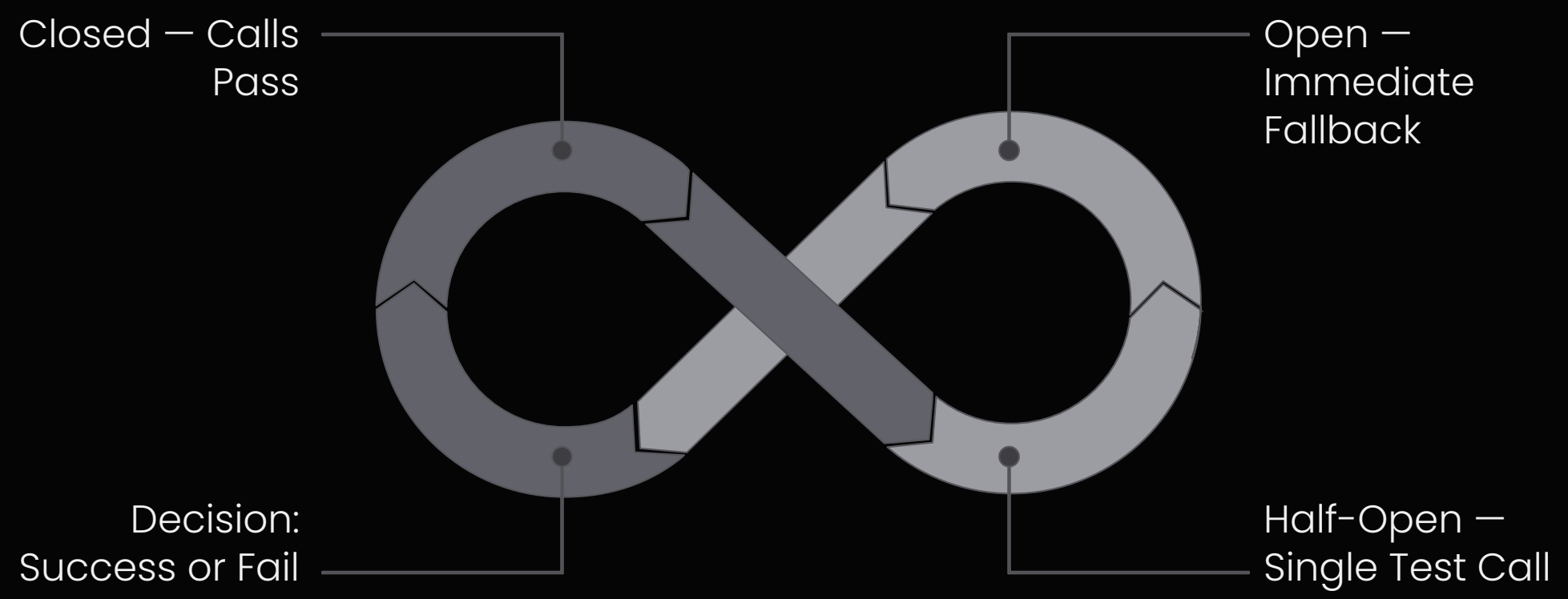
HashiCorp

Kubernetes Service

Service discovery nativo

3.5 Circuit Breaker (Disjuntor)

Se `pagamentos-ms` está lento, `pedidos-ms` pode travar todas as suas threads esperando respostas que nunca chegam — o problema **propaga-se em cascata**. O Circuit Breaker funciona como um disjuntor eléctrico.



Ferramentas

- **Resilience4j** — padrão actual no Spring
- **Hystrix** — legado da Netflix, em manutenção

Exemplo com Resilience4j

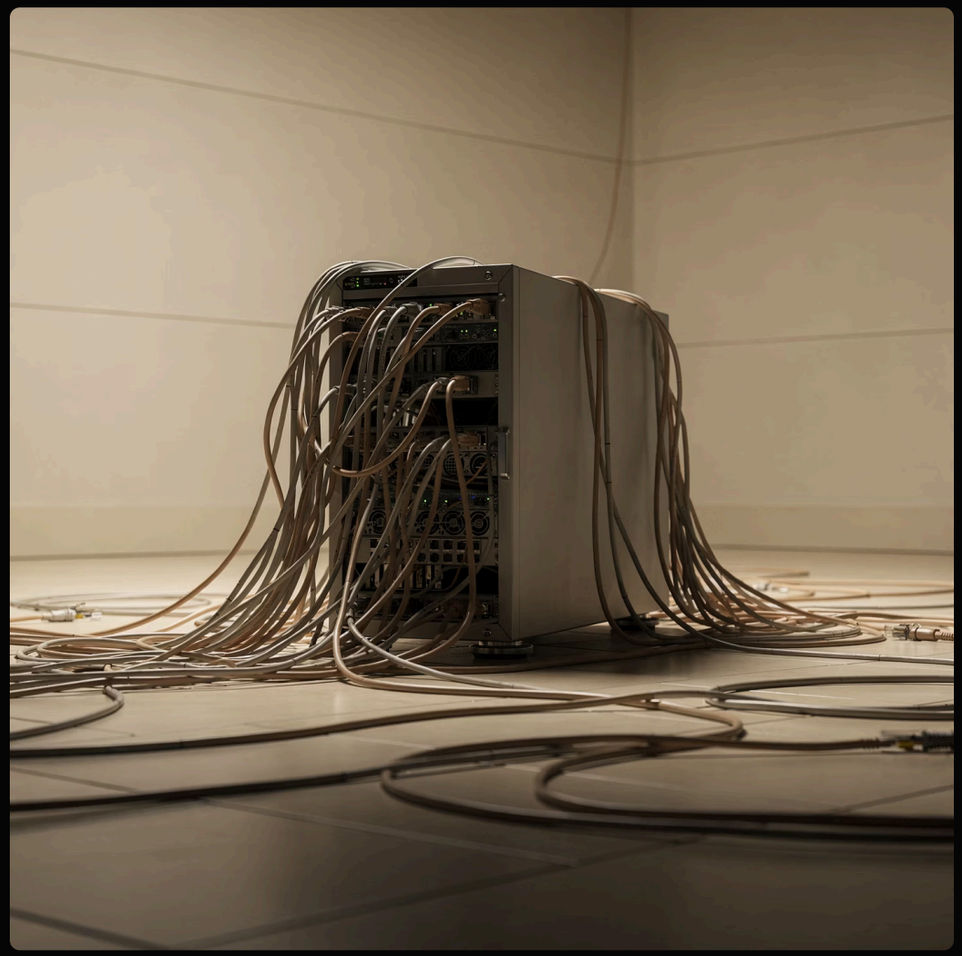
```
@CircuitBreaker(name = "pagamentos",
 fallbackMethod = "pagamentoFallback")
public PagamentoDto processarPagamento(
 PedidoDto pedido) { ... }

public PagamentoDto pagamentoFallback(
 PedidoDto pedido, Exception e) {
 return PagamentoDto.pendente(pedido.getId());
}
```

3.6 Database per Service

Cada microserviço deve ter seu **próprio banco de dados**, acessível apenas por ele. Para obter dados de outro serviço, você precisa chamá-lo pela API — nunca acessar seu banco directamente.

❌ Errado — banco compartilhado

Uma migration no `clientes-ms` pode quebrar o `estoque-ms`. Serviços lêem dados "internos" uns dos outros. Impossível escalar os bancos independentemente.



✅ Correcto — banco por serviço

Cada serviço tem seu próprio banco — inclusive pode usar tecnologias diferentes (MySQL, PostgreSQL) adequadas às suas necessidades. Encapsulamento e independência reais.

3.7 Containerização com Docker

O **Docker** empacota a aplicação e todas as suas dependências numa **imagem** portátil, eliminando o problema de configurar servidores manualmente para dezenas de microserviços.

Conceito	Analogia	O que é
Dockerfile	Receita	Instruções para construir a imagem
Image	Classe	Template imutável com código + dependências
Container	Instância	Processo em execução criado a partir de uma imagem
Registry	NPM/Maven Central	Repositório de imagens (Docker Hub, ECR, GCR)
compose.yaml	docker-compose	Orquestra múltiplos containers localmente

Você já usa o Docker no projecto: o `compose.yaml` que sobe o MySQL é o Docker Compose. O banco de dados do **vendas-ms** roda em um container Docker.

3.8 Orquestração com Kubernetes

O Docker Compose é ótimo para desenvolvimento local, mas em produção você precisa de escala automática, reinício de serviços, distribuição de carga e deploys sem downtime. O **Kubernetes (K8s)** é o orquestrador de containers mais usado em produção.

Conceito	O que é
Pod	Menor unidade — um ou mais containers juntos
Deployment	Define réplicas e como actualizá-las
Service	Endereço estável para um conjunto de pods
Ingress	Ponto de entrada externo — como o API Gateway
ConfigMap/Secret	Configurações e credenciais separadas do código
Namespace	Agrupamento lógico de recursos no cluster

Para o contexto da materia, o importante é entender que o Kubernetes é a plataforma onde microserviços rodam em produção. O `compose.yaml` local é o equivalente simplificado para desenvolvimento.

3.9 Observabilidade

Com dezenas de serviços, quando algo dá errado, como você descobre **onde** está o problema? A observabilidade é composta por três pilares:

Logs

O **quê** aconteceu e quando. Registo de eventos e erros por serviço.

ELK Stack / Loki

Métricas

Quantos / quão rápido. Dados numéricos sobre performance e saúde.

Prometheus / Grafana

Rastreamento Distribuído

Qual **caminho** uma requisição percorreu entre múltiplos serviços.

Jaeger / OpenTelemetry

Distributed Tracing: um `Trace ID` único é gerado na entrada e propagado em todos os headers HTTP ao longo da cadeia. Com ele, você busca num sistema centralizado e vê **toda** a jornada de uma requisição, mesmo que tenha passado por 5 serviços diferentes.

3.10 Consistência Eventual e o Teorema CAP

Em sistemas distribuídos com comunicação assíncrona, dados gravados num serviço **não aparecem imediatamente** nos outros. **Consistência eventual** significa: o sistema *vai* atingir um estado consistente, mas pode não ser imediato.

O Teorema CAP

Um sistema distribuído pode garantir no máximo **dois** dos três:

Propriedade	O que significa
Consistency (C)	Toda leitura vê a escrita mais recente
Availability (A)	Toda requisição recebe uma resposta
Partition tolerance (P)	Sistema funciona mesmo com falhas de rede

A escolha na prática

Em sistemas distribuídos, a falha de rede (P) é **inevitável** — você precisa tolerá-la. A escolha real é entre:

- **CP** — consistência + tolerância, sacrifica disponibilidade
- **AP** — disponibilidade + tolerância, sacrifica consistência imediata

A maioria dos microserviços escolhe **AP** e aceita consistência eventual.

4 — O vendas-ms no contexto de microserviços

Microserviço

A aplicação inteira é um serviço com escopo definido: gestão de vendas.

Bounded Context

Clientes e pedidos no contexto de vendas — com seus próprios campos e regras.

Banco por serviço

MySQL próprio, configurado no `compose.yaml` e `application.properties`.

Comunicação síncrona

OpenFeign chamando a API ViaCEP — um serviço externo.

Containerização

MySQL roda em Docker; a aplicação pode ser containerizada com um Dockerfile.

Autenticação delegada

OAuth2 com GitHub — autenticação como serviço externo, sem gerir senhas.

O que falta para produção?

Resposta

- API Gateway
- Service Discovery
- Circuit Breaker
- Distributed Tracing
- Dockerfile
- Health Check

Quando NÃO usar microserviços

Microserviços não são bala de prata. Eles resolvem problemas que surgem com **escala** — de produto, de times, de tráfego. Antes dessa escala, eles introduzem complexidade sem benefício proporcional.

Sinais de que o monólito ainda é a escolha certa:

- O produto ainda está sendo validado no mercado (não faz sentido otimizar o que pode mudar completamente)
- O time tem menos de ~8 pessoas (o overhead operacional de microserviços consome tempo de desenvolvimento)
- As fronteiras de domínio ainda não estão claras (um microserviço mal dividido é pior que um monólito)

Sinais de que pode ser hora de decompor:

- Times diferentes estão bloqueados uns pelos outros para fazer deploys
- Uma parte da aplicação precisa de escalabilidade muito diferente das outras
- Partes do sistema precisam de tecnologias radicalmente diferentes (ex: ML em Python, API em Java)
- O processo de build e teste já demora mais de 20-30 minutos

Estratégia comum: começar com um monólito bem estruturado (com módulos bem separados internamente) e extrair microserviços conforme a necessidade aparece. Isso é chamado de **"Strangler Fig Pattern"** — o monólito vai sendo gradualmente substituído pelos serviços.